

DOCUMENTACIÓN TÉCNICA

Sistema de Gestión de Reservas y Boletos Aéreos con Apache Cassandra

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Ingeniería en Ciencias y Sistemas

Curso: Sistemas de Bases de Datos 2

Proyecto: Primer Proyecto

Carné: 202202072

1. Introducción

El proyecto implementa una base de datos distribuida para la gestión de pasajeros, aeronaves, vuelos, asientos, reservas y pagos utilizando Apache Cassandra. El diseño físico sigue un enfoque Query-Driven: cada tabla se construye a partir del patrón de acceso que debe resolver, priorizando lecturas directas por clave, desnormalización y ausencia de JOIN y ALLOW FILTERING.

2. Objetivos

- Implementar un clúster Apache Cassandra de tres nodos.
- Aplicar NetworkTopologyStrategy con Replication Factor 3.
- Diseñar tablas Query-Driven para cinco consultas obligatorias.
- Cargar al menos 100,000 reservas mediante Python y Batch Writes.
- Implementar TTL para reservas temporales.
- Evaluar ONE, QUORUM y ALL ante operación normal, caída y recuperación de un nodo.

3. Tecnologías utilizadas

Componente	Versión / uso
Apache Cassandra	4.1.12
Docker / Docker Compose	Clúster reproducible de 3 nodos
Python	3.12
cassandra-driver	3.30.1
Faker	40.39.0
Git / GitHub	Control de versiones y entrega

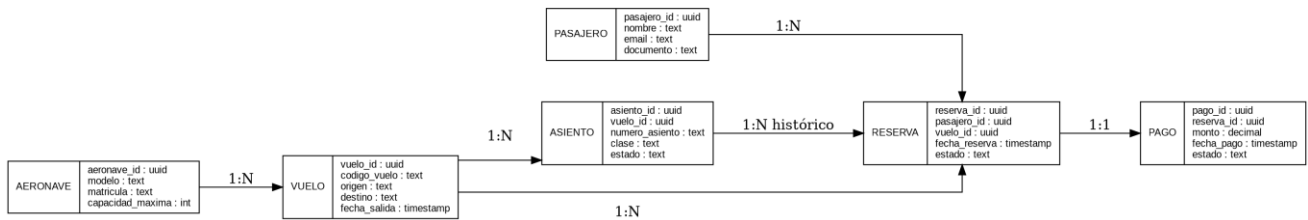
4. Arquitectura del clúster

El entorno está definido en docker-compose.yml y contiene tres nodos: cassandra1, cassandra2 y cassandra3. Todos pertenecen al clúster AerolineCluster, al datacenter datacenter1 y al rack rack1. En el host se publican los puertos 9042, 9043 y 9044 respectivamente.

Cluster Name: AerolineCluster
Datacenter: datacenter1
Rack: rack1
Snitch: GossipingPropertyFileSnitch
Partitioner: Murmur3Partitioner
Nodos Live: 3
Unreachable: 0

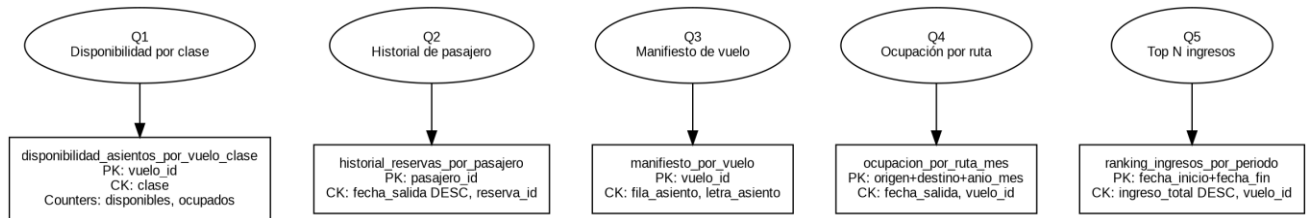
5. Modelo conceptual

El modelo conceptual representa las entidades del dominio y sus relaciones principales. En Cassandra este modelo sirve como referencia de negocio, pero no se traduce de manera directa a tablas relacionales normalizadas.



6. Diseño Query-Driven

El diseño lógico parte de las cinco consultas requeridas y crea una tabla específica para cada patrón de acceso. La duplicación de datos es intencional y evita JOIN, búsquedas por columnas no indexadas y ALLOW FILTERING.



7. Keyspace y replicación

```

CREATE KEYSPACE IF NOT EXISTS aerolinea
WITH replication = {
    'class': 'NetworkTopologyStrategy',
    'datacenter1': 3
}
AND durable_writes = true;
  
```

Con Replication Factor 3, cada dato del keyspace aerolinea posee una réplica en cada nodo del datacenter. nodetool status aerolinea mostró ownership efectivo de 100% en los tres nodos.

8. Tablas del sistema

8.1 Tablas operativas

```

pasajeros_por_id
aeronaves_por_id
vuelos_por_id
asientos_por_vuelo
reservas_por_id
pagos_por_reserva
  
```

8.2 Tablas Query-Driven

Consulta	Tabla	Clave principal
Q1	disponibilidad_asientos_por_vuelo_clase	PK vuelo_id; CK clase
Q2	historial_reservas_por_pasajero	PK pasajero_id; CK fecha_salida, reserva_id
Q3	manifiesto_por_vuelo	PK vuelo_id; CK fila_asiento, letra_asiento

Q4	ocupacion_por_ruta_mes	PK origen+destino+anio_mes; CK fecha_salida, vuelo_id
Q5	ranking_ingresos_por_periodo	PK fecha_inicio+fecha_fin; CK ingreso_total DESC, vuelo_id

9. Justificación de claves por consulta

9.1 Q1 - Disponibilidad por clase

La partition key vuelo_id concentra en una sola partición las tres clases de un vuelo. Los campos disponibles y ocupados son counters, evitando COUNT en tiempo de consulta.

9.2 Q2 - Historial del pasajero

La partition key pasajero_id agrupa el historial del pasajero. fecha_salida es la primera clustering column y permite rangos con >= y <=, manteniendo orden descendente.

9.3 Q3 - Manifiesto del vuelo

La partition key vuelo_id recupera todo el manifiesto en una sola lectura. fila_asiento y letra_asiento se separan para conservar orden numérico real: 1A, 1B, ..., 10A, en lugar de orden lexicográfico.

9.4 Q4 - Ocupación por ruta

La partition key compuesta origen+destino+anio_mes implementa bucketing mensual. reservas_confirmadas y capacidad_total se almacenan como contadores por vuelo, permitiendo calcular el porcentaje sin recorrer reservas individuales.

9.5 Q5 - Top N por ingresos

La partición materializa un período conocido y ordena por ingreso_total DESC. LIMIT N devuelve directamente los primeros vuelos sin ordenar resultados en Python.

10. Carga masiva

La carga definitiva se ejecutó con scripts/03_carga_masiva.py. Los datos se generaron de forma determinista con Faker y UUID v5.

Dato	Cantidad
Reservas	100,000
Pagos	100,000
Historial Q2	100,000
Manifiesto Q3	100,000
Pasajeros	5,000
Aeronaves	20
Vuelos	1,000
Asientos	120,000
Filas Q1	3,000
Filas Q4	1,000
Filas Q5	1,000

La distribución de reservas fue de 80,000 confirmadas, 10,000 pendientes y 10,000 canceladas. La ejecución final tomó 26.44 segundos en el equipo de pruebas.

11. Batch Writes

Se utilizaron BatchStatement UNLOGGED de cinco reservas. Cada reserva genera cuatro escrituras: reservas_por_id, pagos_por_reserva, historial_reservas_por_pasajero y manifiesto_por_vuelo. Por tanto, cada batch contiene 20 statements y la carga de 100,000 reservas produjo 20,000 Batch Writes.

```
BatchStatement(  
    batch_type=BatchType.UNLOGGED,  
    consistency_level=ConsistencyLevel.ONE  
)
```

12. Consultas obligatorias y resultados

12.1 Q1 - Disponibilidad por clase

```
SELECT clase, disponibles, ocupados  
FROM disponibilidad_asientos_por_vuelo_clase  
WHERE vuelo_id = 7738e6e4-6f78-5a6a-9adc-1f33c41fe130;
```

Clase	Disponibles	Ocupados
BUSINESS	2	10
PREMIUM	1	17
ECONOMY	27	63

12.2 Q2 - Historial por rango

Para el pasajero af65cd22-ae00-5a4a-8268-45d22029b352 entre 2026-01-10 y 2026-01-26 se obtuvieron 13 registros, ordenados de la fecha más reciente a la más antigua.

12.3 Q3 - Manifiesto de vuelo

Para el vuelo 7738e6e4-6f78-5a6a-9adc-1f33c41fe130 se obtuvieron 100 filas en orden de asiento, comenzando 1A, 1B, 1C y continuando correctamente hasta las filas de dos dígitos.

12.4 Q4 - Ocupación por ruta

Ruta / período	Confirmadas	Capacidad	Ocupación
GUA -> MEX / enero 2026	1,280	1,920	66.6667%

12.5 Q5 - Top N de ingresos

La consulta de enero 2026 devolvió 10 vuelos mediante ORDER BY ingreso_total DESC LIMIT 10. En los datos generados existieron empates de Q19,200.00 por vuelo debido a la distribución uniforme de clases y estados.

13. TTL

Se insertó una reserva temporal con USING TTL 30. La consulta TTL(estado) mostró 22 segundos y luego 11 segundos restantes. Tras transcurrir el TTL, la búsqueda devolvió 0 filas, validando la expiración automática.

```
INSERT INTO reservas_por_id (...)  
VALUES (...)  
USING TTL 30;
```

14. Consistency Levels

Con RF=3, ONE requiere una réplica, QUORUM requiere dos y ALL requiere las tres. Cada nivel se probó 20 veces sobre la misma consulta.

14.1 Tres nodos activos

Nivel	Éxitos	Errores	Mínima	Promedio	Máxima
ONE	20	0	15.781 ms	17.041 ms	21.171 ms
QUORUM	20	0	7.487 ms	16.109 ms	25.148 ms
ALL	20	0	9.179 ms	16.960 ms	28.605 ms

14.2 Un nodo caído

Nivel	Éxitos	Errores	Mínima	Promedio	Máxima
ONE	20	0	6.567 ms	15.107 ms	21.327 ms
QUORUM	20	0	10.802 ms	16.361 ms	23.195 ms
ALL	0	20	8.916 ms	15.749 ms	22.702 ms

ALL falló con el mensaje: "Cannot achieve consistency level ALL; required_replicas: 3; alive_replicas: 2". El comportamiento es el esperado.

14.3 Clúster recuperado

Nivel	Éxitos	Errores	Mínima	Promedio	Máxima
ONE	20	0	12.334 ms	16.667 ms	21.832 ms
QUORUM	20	0	12.785 ms	15.671 ms	17.908 ms
ALL	20	0	14.424 ms	16.481 ms	18.664 ms

Después de iniciar nuevamente cassandra3, los tres nodos regresaron a UN y ALL volvió a responder satisfactoriamente.

15. Resultados generales

Indicador	Resultado
Nodos Cassandra	3
Versión	4.1.12
Replication Factor	3
Estrategia	NetworkTopologyStrategy
Tablas	11
Reservas	100,000
Batch Writes	20,000
Consultas requeridas	5/5
JOIN	0
ALLOW FILTERING	0
TTL	Validado
Falla y recuperación	Validada

16. Organización del código

Primer Proyecto/

|— cql/

| |— 01_keyspace.cql

| |— 02_schema.cql

| |— 03_queries.cql

|— scripts/

| |— 01_verificar_conexion.py

- | |— 02_carga_prueba.py
- | |— 03_carga_masiva.py
- | |— 04_prueba_tolerancia.py
- |— diagramas/
- |— docs/
- |— evidencias/
- |— docker-compose.yml
- |— requirements.txt

17. Conclusiones

El proyecto demostró un modelo Cassandra orientado a consultas, con desnormalización intencional y acceso directo por partition keys. La carga masiva de 100,000 reservas se completó con Batch Writes y las cinco consultas obligatorias funcionaron sin JOIN ni ALLOW FILTERING.

La configuración NetworkTopologyStrategy con RF=3 mantuvo tres copias de los datos. Las pruebas mostraron que ONE y QUORUM continúan disponibles ante la caída de un nodo, mientras que ALL requiere las tres réplicas. Al recuperar el nodo, ALL volvió a funcionar, comprobando el comportamiento esperado del clúster.